



Final Presentation

Group 7 | Bolislis, Del Pilar, Salto, See





Content Overview

01

**Recap of Status
Report**

02

**Changes and
Improvements**

03

Final Evaluation

04

Insights and Reflection





01

Recap of Status Report



Recap: Problem Statement

A popular retail chain is launching a limited-time VIP membership offer as part of its holiday sale campaign, promoted exclusively through phone calls to existing customers.

However, this method is expensive and inefficient, as many contacted customers are unlikely to be interested. The lack of a data-driven approach to identify high-potential buyers leads to wasted efforts and makes it difficult to optimize sales strategies.





Context of the Problem

A popular retail chain is preparing for a big holiday sale campaign that targets its existing customers.

As part of the campaign, the company plans to launch an exclusive VIP membership that gives a 20% discount on all purchases for a year which will be offered for a limited time at \$499 (regular price: \$999).

The offer will be promoted exclusively to current customers via phone call campaigns.



Pain Points to Address

High Marketing Costs

The reliance on phone calls in reaching out to all customers makes the process expensive and inefficient

Low Conversion Rates

Not many customers contacted will be interested in the VIP membership which leads to wasted efforts on unlikely buyers

Lack of Customer Insights

Lacks a data-driven approach to identify highly potential customers which makes it difficult to optimize sales efforts



Objectives



Business Goal

The company aims to optimize its marketing strategy by implementing a more targeted, data-driven approach. By focusing on customers most likely to purchase the VIP membership, the goal is to increase campaign success rates while improving the efficiency and effectiveness of marketing efforts.



Data Mining Goal

To support the business goal, the company aims to develop a predictive model designed to:

1. Predict the probability of a customer accepting the VIP membership offer by generating a likelihood score for each individual, and
2. Identify key factors that influence a customer's decision to accept the offer, such as spending habits, purchase frequency, and customer demographics.

Context of the Dataset

Last year, a marketing campaign aimed at promoting memberships collected data on customer responses. To achieve our goal of predicting the likelihood that a customer will accept or decline a membership offer, we are using this dataset as the basis for our analysis.

Understanding the context of this dataset is crucial, as it helps interpret the target variable and properly guide the data cleaning, preprocessing, and modeling process.





Overview of Features and Target Variable

The dataset contains provides a well-rounded view of customer profiles and behavior, covering:

- **Customer Demographics:** Includes details about background, education, income, and family composition.
 - **Customer Loyalty & Recency:** Captures when customers joined and how recently they engaged, offering insights into loyalty and activity levels.
 - **Spending Behavior:** Tracks how much customers spend across different product categories, helping to identify preferences and purchasing power.
 - **Purchase Preferences:** Reflects how customers shop through deals, online, catalogs, in-store visits, and their level of online activity.
 - **Customer Satisfaction:** indicates whether a customer has expressed dissatisfaction, showcasing their overall experience.
 - **Target Variable:** Represents whether the customer responded to the most recent marketing campaign.
- 



Pipeline Summary

01

Data Description

02

Data Cleaning

03

Exploratory Data Analysis

04

Data Preprocessing

05

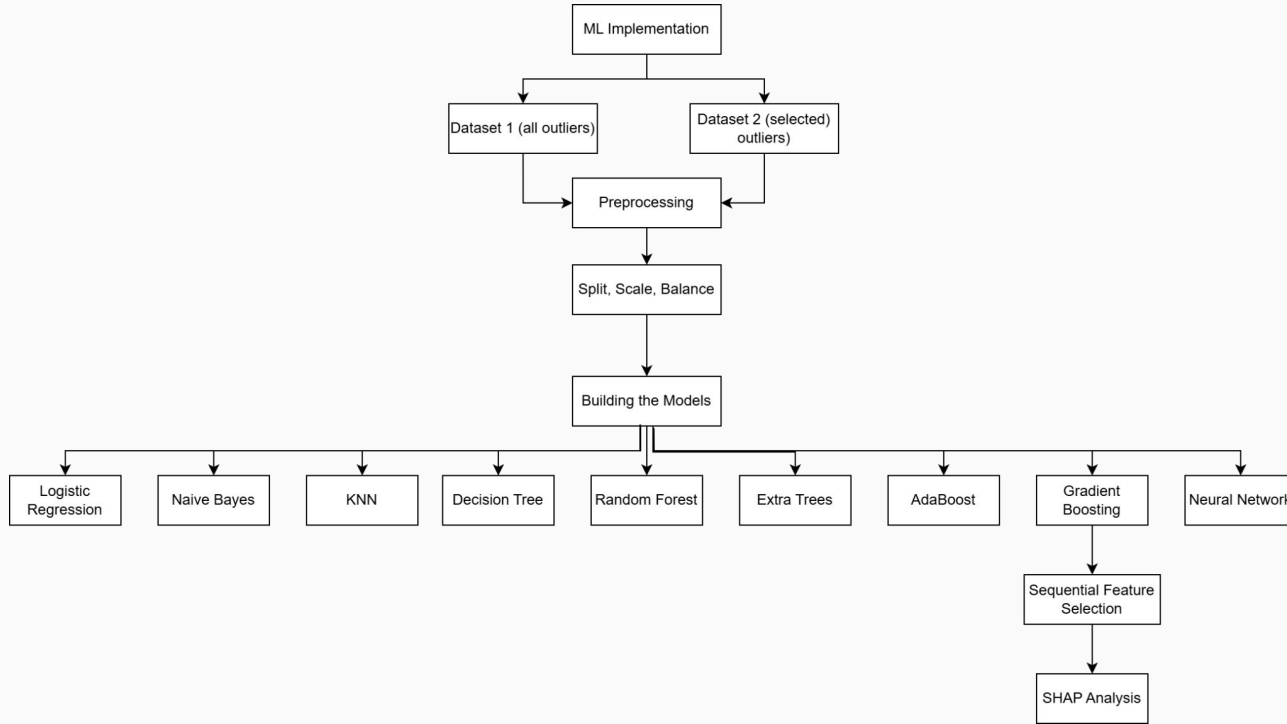
Model Building

06

Model Evaluation



Pipeline Diagram



Insights from Initial EDA

Early exploratory data analysis revealed key structural and quality aspects of the dataset that will shape preprocessing:

- The dataset contains **2240 rows** and **22 columns**: 18 columns are integers, 1 is a float (or continuous), and 3 string-based columns.
- Includes a mix of **numerical and categorical variables**.
- **24 missing values**, were found in the **'Income' column**, requiring imputation.
- Despite being numerically encoded, the **'Complain' and 'Response' columns** are **binary categorical** variables with values of 0 and 1.
- The **'Dt_customer' column** is **not in proper datetime format** and must be converted.
- **No duplicate rows** were found,
- For consistency:
 - In the **'Education' column**, **"2n Cycle"** should be **grouped under Masters**, **"Graduation"** should be **renamed to Bachelors**, and **"Master"** to **Masters**.
 - In the **'Marital_Status' column**, labels like **Alone**, **YOLO**, and **Absurd** should be grouped with **Single** for standardization.



Data Cleaning

This is a crucial step before the exploratory data analysis (EDA) to **ensure the dataset is accurate and reliable**. It addresses errors, inconsistencies, and missing values, preventing misleading insights and distorted visualizations, causing incorrect conclusions during EDA. Clean data ensures features are properly formatted, like standardizing categories or converting data types, which enhances accuracy during analysis and supports effective modeling.

By resolving issues early, **data cleaning saves time, highlights true patterns, and builds a strong foundation for reliable insights and predictive models.**



Dropping the ID Column

	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer
0	1970	Graduation	Divorced	84835.0	0	0	6/16/2014
1	1961	Graduation	Single	57091.0	0	0	6/15/2014
2	1958	Graduation	Married	67267.0	0	1	5/13/2014
3	1967	Graduation	Together	32474.0	1	1	11/5/2014
4	1989	Graduation	Single	21474.0	1	0	8/4/2014

The ID column, being a **non-informative unique identifier**, was removed to keep the dataset clean and focused on relevant features.

Handling Missing Values

```
[ ] cleaned_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 22 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Id                     2240 non-null   int64
1   Year_Birth             2240 non-null   int64
2   Education              2240 non-null   object
3   Marital_Status         2240 non-null   object
4   Income                 2240 non-null   float64
5   Kidhome                2240 non-null   int64
6   Teenhome               2240 non-null   int64
7   Dt_Customer            2240 non-null   object
8   Recency                2240 non-null   int64
9   MntWines               2240 non-null   int64
10  MntFruits              2240 non-null   int64
11  MntMeatProducts        2240 non-null   int64
12  MntFishProducts        2240 non-null   int64
13  MntSweetProducts       2240 non-null   int64
14  MntGoldProds           2240 non-null   int64
15  NumDealsPurchases      2240 non-null   int64
16  NumWebPurchases        2240 non-null   int64
17  NumCatalogPurchases    2240 non-null   int64
18  NumStorePurchases      2240 non-null   int64
19  NumWebVisitsMonth       2240 non-null   int64
20  Response               2240 non-null   int64
21  Complain               2240 non-null   int64
dtypes: float64(1), int64(18), object(3)
memory usage: 385.1+ KB
```

No missing values already in Income column.

To prevent bias in predictive models and its results, missing income values were **imputed** using the average income.

Fixing Data Types

```
[ ] cleaned_df['Dt_Customer']
```



Dt_Customer

0	2014-06-16
1	2014-06-15
2	2014-05-13
3	2014-11-05
4	2014-08-04
...	...
2235	2013-07-03
2236	2013-01-22
2237	2012-03-12
2238	2012-11-29
2239	2012-01-09

2240 rows × 1 columns

dtype: datetime64[ns]

The **Dt_Customer** column , initially stored as text (object type), was **converted to a datetime format to enable time-based calculations** .

Fixing Data Types

	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Enrollment_Duration
0	1970	Graduation	Divorced	84835.0	0	0	3851
1	1961	Graduation	Single	57091.0	0	0	3852
2	1958	Graduation	Married	67267.0	0	1	3885
3	1967	Graduation	Together	32474.0	1	1	3709
4	1989	Graduation	Single	21474.0	1	0	3802

The 'Dt_Customer' column was converted to a proper datetime format, enabling the **calculation of a new feature**, **Enrollment_Duration**, which represents the number of days each customer has been with the company as of December 31, 2024 (post-campaign data collection date).

This transformation replaced the original date column with a numerical feature to avoid issues with correlation and modeling. The **'Dt_Customer' column was then dropped** to remove redundancy.

This attribute can help us understand customer retention and engagement over time — essential for analyzing loyalty and predicting future behaviors.

Fixing Labels

```
[ ] 1 # Apply the mappings to the respective columns
    2 cleaned_df['Marital_Status'] = cleaned_df['Marital_Status'].replace(marital_status_mapping)
    3 cleaned_df['Education'] = cleaned_df['Education'].replace(education_mapping)
```

```
[ ] 1 cleaned_df['Marital_Status'].value_counts().reset_index()
```

	Marital_Status	count
0	Married	884
1	Together	580
2	Single	487
3	Divorced	232
4	Widow	77

```
[ ] 1 cleaned_df['Education'].value_counts().reset_index()
```

	Education	count
0	Bachelors	1127
1	Masters	573
2	PhD	488
3	Basic	64

Inconsistent labels in '**Marital_Status**' (e.g., "YOLO," "Alone," "Absurd") and '**Education**' (e.g., "Graduation," "Master," "2n Cycle") were **standardized using value mappings** to ensure consistent categories for accurate analysis and modeling.

The dataset contains **2,240 customer records** which were all cleaned and completed with no missing values.

	Year_Birth	Income	Kidhome	Teenhome	Enrollment_Duration	Recency
count	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000

MntWines	MntFruits	MntMeatProducts	MntFishProducts	MntSweetProducts	MntGoldProds
2240.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000

[illegible]

EDA - Descriptive Statistics

Year_Birth

	Year_Birth
count	2240.000000
mean	1968.805804
std	11.984069
min	1893.000000
25%	1959.000000
50%	1970.000000
75%	1977.000000
max	1996.000000

Mean birth year is 1968 (average age: mid-50s). The **IQR** is from 1959 to 1977 and the **Range** is 1893 to 1996, indicating the presence of outliers.

Income

	Income
count	2240.000000
mean	52247.251354
std	25037.797168
min	1730.000000
25%	35538.750000
50%	51741.500000
75%	68289.750000
max	666666.000000

Mean income is \$52,247, with a **max** value of \$666,666 suggesting outliers. The **median** is \$51,741, with an **IQR** of \$35,539–\$68,290, indicating moderate right skew.

EDA - Descriptive Statistics

Kidhome and Teenhome

	Kidhome	Teenhome
count	2240.000000	2240.000000
mean	0.444196	0.506250
std	0.538398	0.544538
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	1.000000	1.000000
max	2.000000	2.000000

Most customers have **no kids or teens at home**, with **max values of 2** for both features, reflecting an older customer base.

Enrollment_Duration

	Enrollment_Duration
count	2240.000000
mean	4190.043304
std	232.229893
min	3678.000000
25%	4018.750000
50%	4191.000000
75%	4363.250000
max	4741.000000

Median of 4,191 days (~11.5 years), showing long-term customer engagement.

EDA - Descriptive Statistics

Recency

	Recency
count	2240.000000
mean	49.109375
std	28.962453
min	0.000000
25%	24.000000
50%	49.000000
75%	74.000000
max	99.000000

Mean recency of **49 days** with a wide **range from 0 to 99 days** , indicating varying engagement levels.

EDA - Descriptive Statistics

Spending Behavior

	MntWines	MntFruits	MntMeatProducts	MntFishProducts	MntSweetProducts	MntGoldProds
count	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000
mean	303.935714	26.302232	166.950000	37.525446	27.062946	44.021875
std	336.597393	39.773434	225.715373	54.628979	41.280498	52.167439
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	23.750000	1.000000	16.000000	3.000000	1.000000	9.000000
50%	173.500000	8.000000	67.000000	12.000000	8.000000	24.000000
75%	504.250000	33.000000	232.000000	50.000000	33.000000	56.000000
max	1493.000000	199.000000	1725.000000	259.000000	263.000000	362.000000

Spending is skewed across all product types. **Wine** has the highest average spending (\$304) with max value of \$1,493, while categories like **Meat**, **Fish**, and **Gold** show high variability and outliers.

EDA - Descriptive Statistics

Purchase Preferences

	NumDealsPurchases	NumWebPurchases	NumCatalogPurchases	NumStorePurchases	NumWebVisitsMonth
count	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000
mean	2.325000	4.084821	2.662054	5.790179	5.316518
std	1.932238	2.778714	2.923101	3.250958	2.426645
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	1.000000	2.000000	0.000000	3.000000	3.000000
50%	2.000000	4.000000	2.000000	5.000000	6.000000
75%	3.000000	6.000000	4.000000	8.000000	7.000000
max	15.000000	27.000000	28.000000	13.000000	20.000000

Store and web purchases are the dominant channels, with **catalog** and **deal purchases** showing less engagement but some heavy users.

EDA - Descriptive Statistics

Response and Complaint Rates

	Response	Complain
count	2240.000000	2240.000000
mean	0.149107	0.009375
std	0.356274	0.096391
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	0.000000	0.000000
max	1.000000	1.000000

Response rate is low : only ~15% accepted the offer, indicating challenges in targeting. Meanwhile, **complaints are rare** : only ~0.9% filed complaints in the last two years, suggesting general customer satisfaction.

EDA - Descriptive Statistics

Key Patterns and Outliers Summary

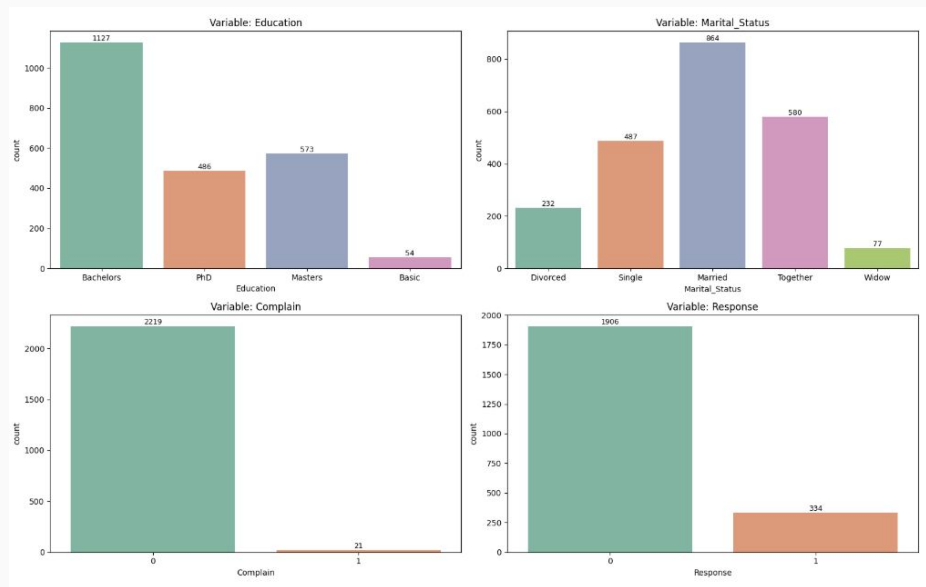
- **Customer Base**
 - Mature, with long-term loyalty and diverse spending behaviors.
- **Engagement**
 - Mixed levels of engagement, with some customers highly active and others less engaged.
- **Income and Spending**
 - Significant outliers in income and spending across categories like wine and gold products.
- **Targeting Challenges**
 - The low acceptance rate of the campaign offer emphasizes the need for more accurate customer targeting.
- **Feature Variability**
 - Notable variability in spending and purchase behavior; requires careful handling for analysis.

EDA - Distribution Analysis

Categorical Variables

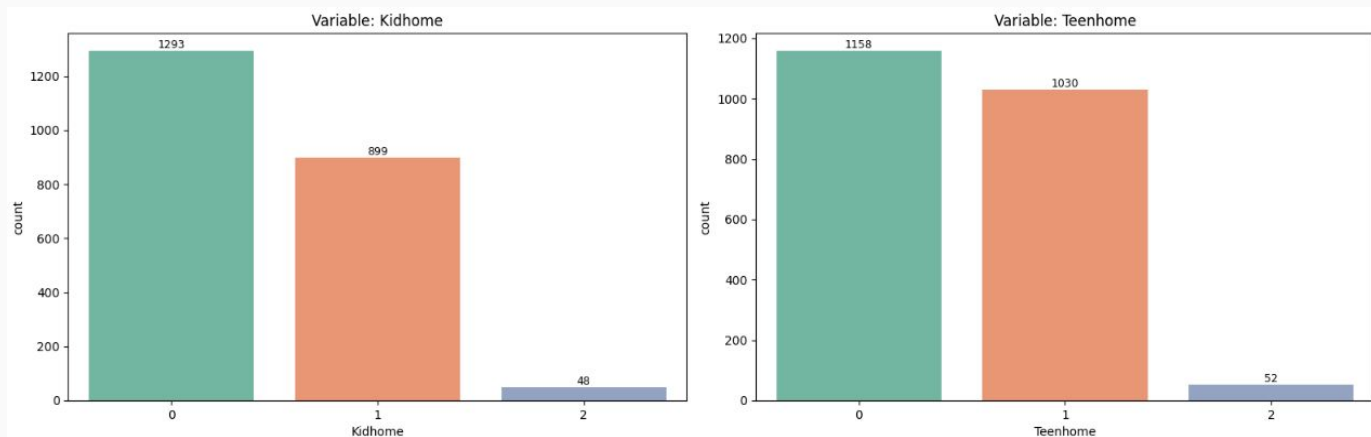
Most customers are **married with a Bachelor's degree**, indicating a **mature, educated demographic**.

Low complaints reflect **high satisfaction**, but a **low response rate** suggests **limited engagement** with last year's campaign.



Distribution Analysis

Discrete Numeric Variables



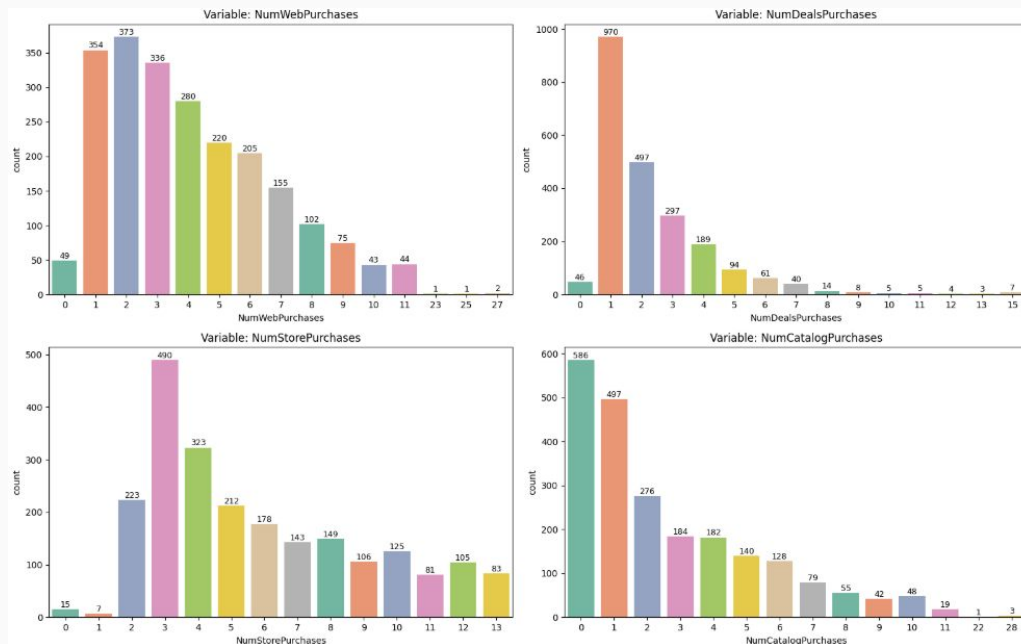
Most customers have **no children or only one child** , with few having two, indicating a majority of **smaller families** .

Distribution Analysis

Discrete Numeric Variables

Most customers made **2–3 purchases or fewer** across all channels, with **store purchases dominating**.

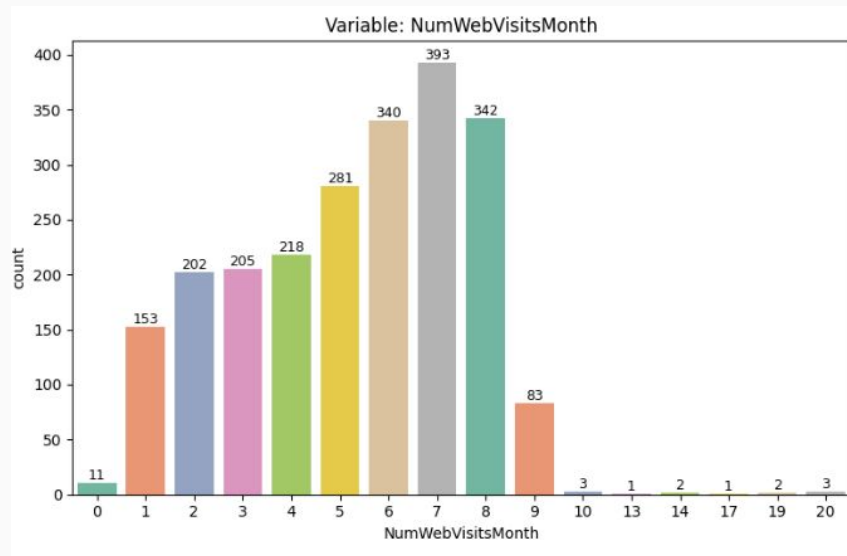
Meanwhile, **catalog and deal-based purchases were rare**, appealing to a smaller, highly engaged subset of customers.



Distribution Analysis

Discrete Numeric Variables

Most customers visited the website **5–8 times in the last month**, with **peaks at 6, 7, and 8 visits**, indicating **consistent engagement**. Few customers had very low (0–1) or high (10+) visit counts, suggesting **rare extremes in activity**.

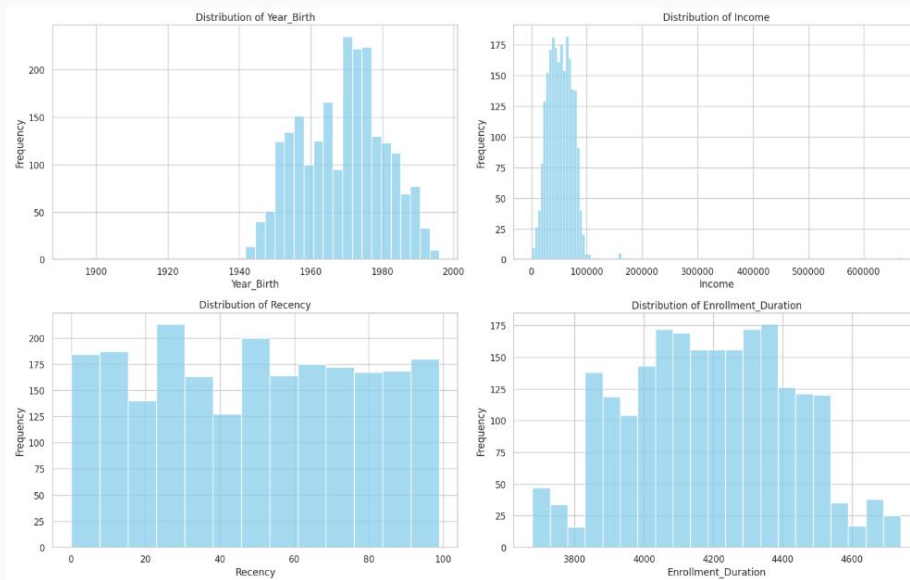


Distribution Analysis

Continuous Numeric Variables

Most customers were born between 1955 and 1980, indicating a **middle-aged demographic with lower to mid-income levels and a few high earners**.

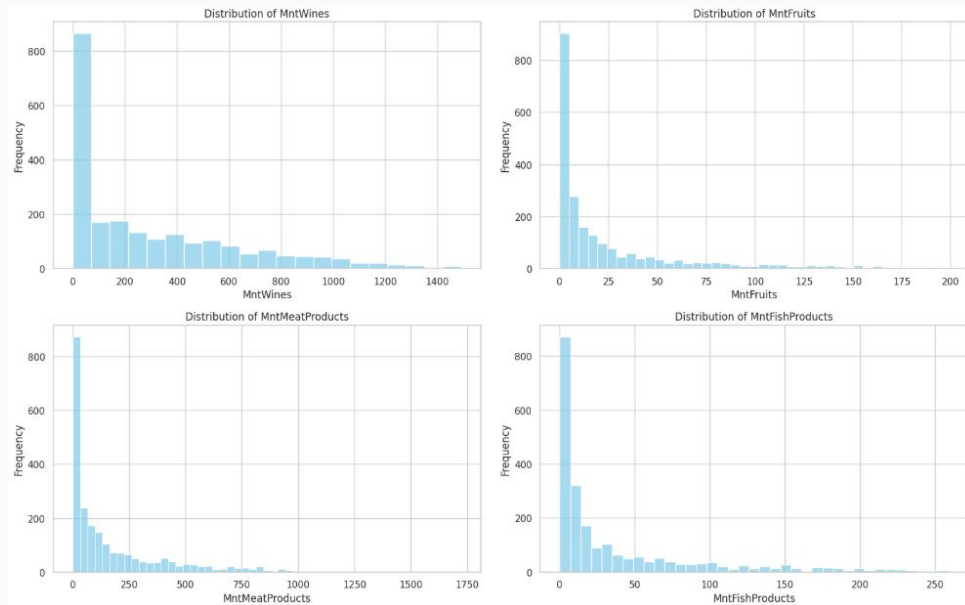
Customer activity is evenly distributed, with recent and less active buyers, while most have been enrolled for 10 to 12 years, reflecting a **stable, long-standing customer base**.



Distribution Analysis

Continuous Numeric Variables

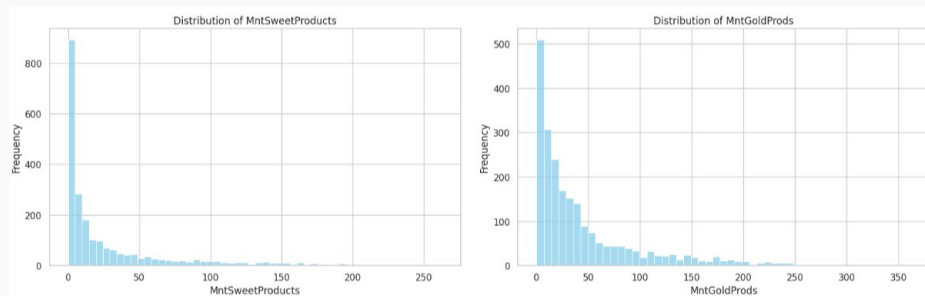
Customer spending across all product categories is **heavily right-skewed**, with **most spending modestly and a few high spenders**. This indicates a small segment of premium or loyal customers driving higher expenditures.



Distribution Analysis

Continuous Numeric Variables

Spending on sweet and gold products is also **heavily right-skewed**, with **most spending little or none and a small group spending more**, suggesting these items appeal to a niche of higher-spending customers or specialized customers.



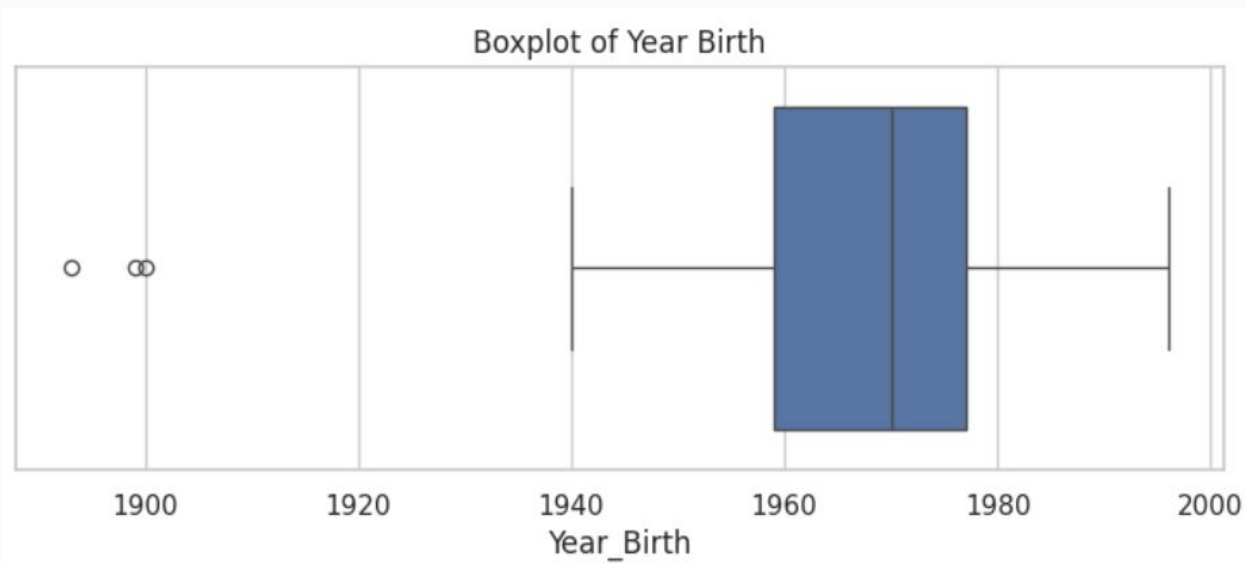
Distribution Analysis

The following columns were identified as having significant outliers based on their boxplots:

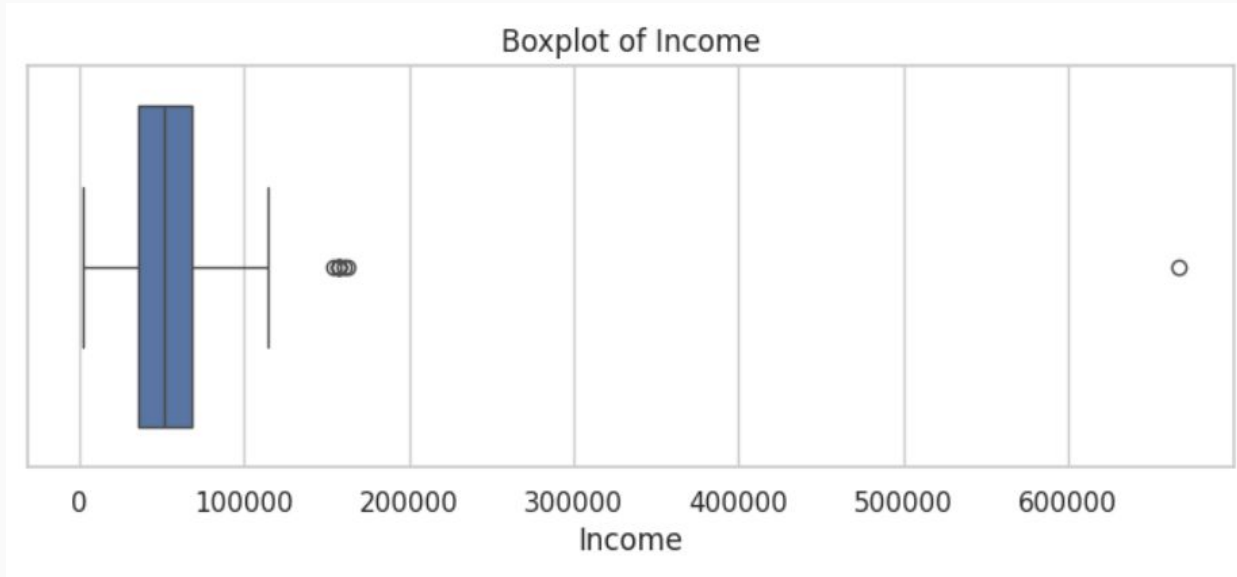
Year_Birth, Income, MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds, NumDealsPurchases, NumWebPurchases, NumCatalogPurchases, NumStorePurchases, and **NumWebVisitsMonth.**

These outliers represent values that fall far outside the typical range of the data, which can **distort distributions** , **skew summary statistics** , and **negatively impact model performance** . Addressing these outliers is essential to ensure that the model learns from representative patterns and does not overfit to extreme or rare values.

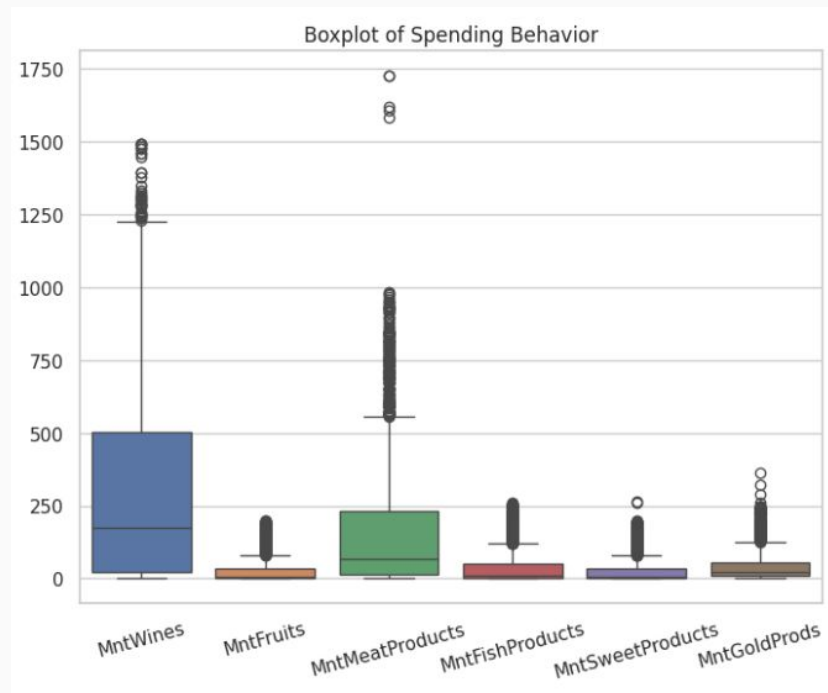
Distribution Analysis



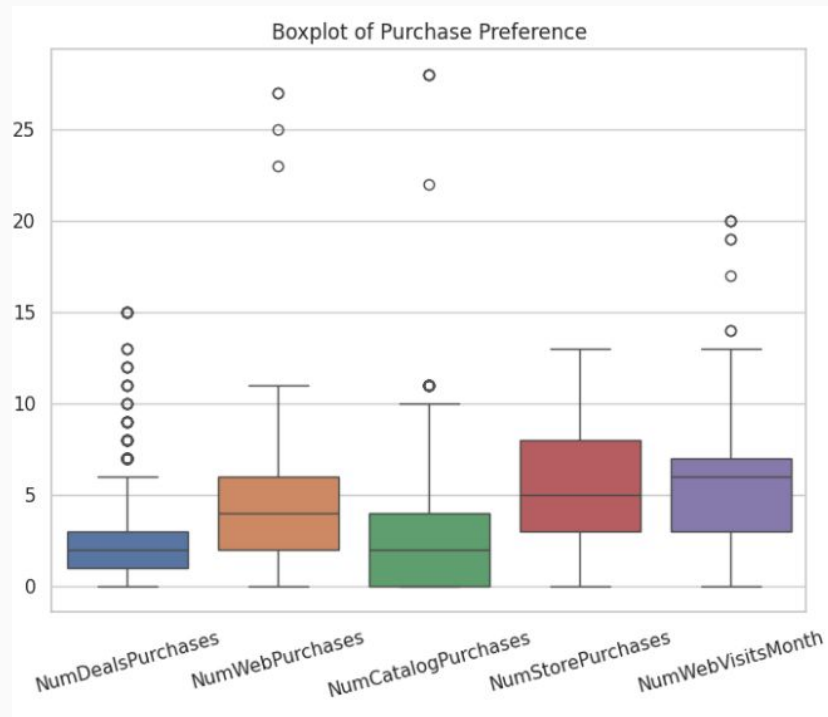
Distribution Analysis



Distribution Analysis



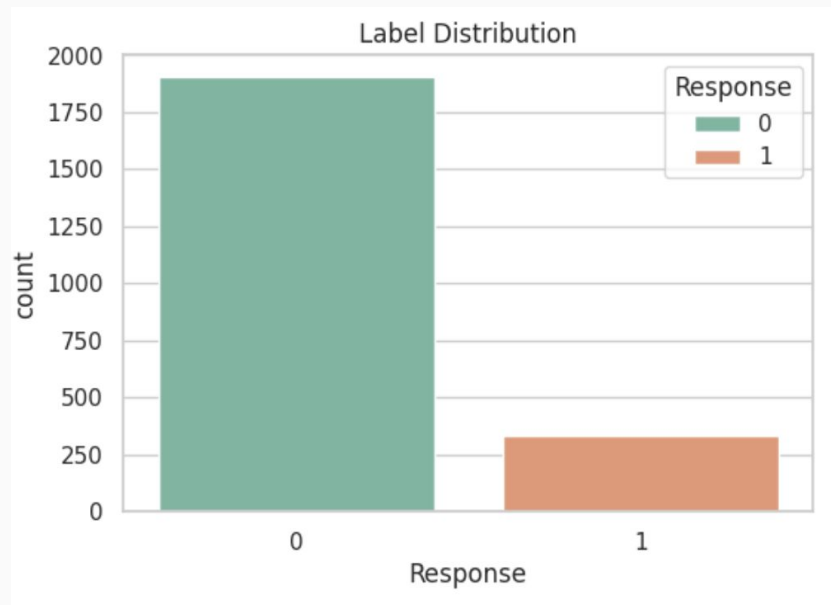
Distribution Analysis



Label Distribution

The **target variable, Response**, is **heavily imbalanced**, with 85% non-responders and only 15% responders to the marketing campaign.

This could lead models to favor non-responders, requiring resampling to ensure balanced predictions.



Encoding Categorical Data (One-hot

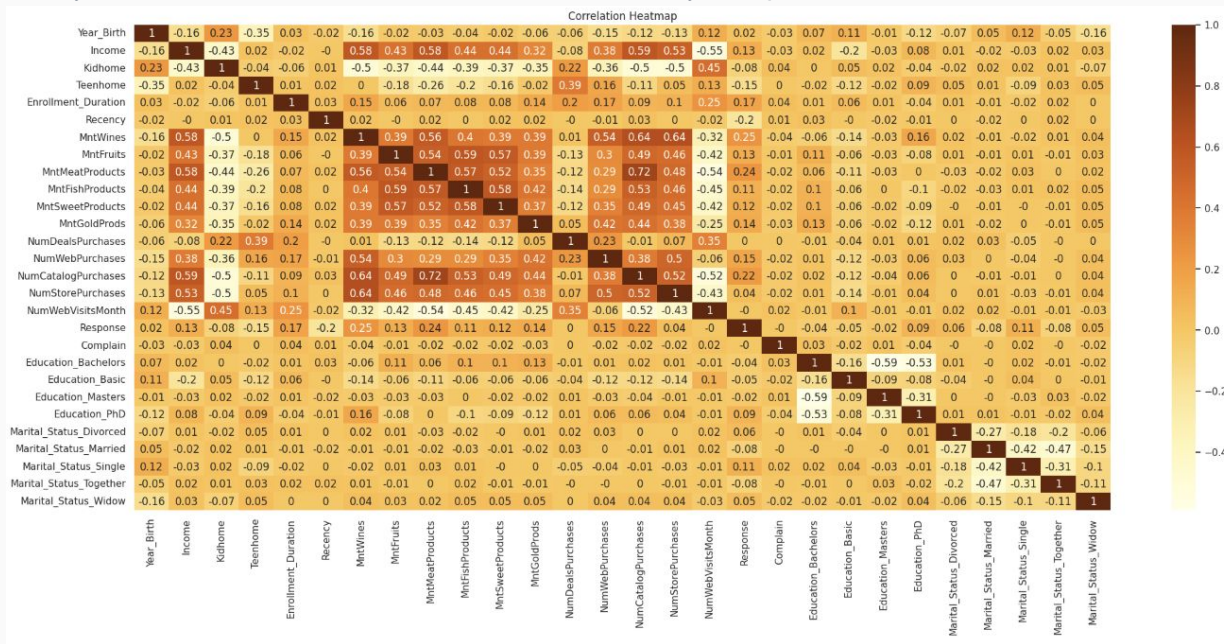
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 28 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Year_Birth             2240 non-null   int64
1   Income                 2240 non-null   float64
2   Kidhome                 2240 non-null   int64
3   Teenhome               2240 non-null   int64
4   Enrollment_Duration    2240 non-null   int64
5   Recency                 2240 non-null   int64
6   MntWines                2240 non-null   int64
7   MntFruits               2240 non-null   int64
8   MntMeatProducts        2240 non-null   int64
9   MntFishProducts        2240 non-null   int64
10  MntSweetProducts       2240 non-null   int64
11  MntGoldProds            2240 non-null   int64
12  NumDealsPurchases      2240 non-null   int64
13  NumWebPurchases        2240 non-null   int64
14  NumCatalogPurchases    2240 non-null   int64
15  NumStorePurchases      2240 non-null   int64
16  NumWebVisitsMonth       2240 non-null   int64
17  Response                2240 non-null   int64
18  Complain                2240 non-null   int64
19  Education_Bachelors     2240 non-null   bool
20  Education_Basic         2240 non-null   bool
21  Education_Masters       2240 non-null   bool
22  Education_PhD           2240 non-null   bool
23  Marital_Status_Divorced 2240 non-null   bool
24  Marital_Status_Married  2240 non-null   bool
25  Marital_Status_Single   2240 non-null   bool
26  Marital_Status_Together 2240 non-null   bool
27  Marital_Status_Widow    2240 non-null   bool
dtypes: bool(9), float64(1), int64(18)
memory usage: 352.3 KB
```

One-hot encoding, typically a preprocessing step, was applied early in this case to support correlation analysis by converting the categorical variables, 'Marital Status' and 'Education' into **binary columns**.

This step allowed categorical features to be included in the correlation analysis, enabling a deeper **exploration of their relationships with other variables** and yielding more meaningful insights from the data.

Correlation Analysis

Through correlation analysis, we can measure the strength and direction of relationships between numerical features and the target variable. It helps identify influential predictors and detect multicollinearity, using a correlation matrix for visualization.





Correlation Analysis

The target variable Response represents whether a customer accepted the offer from last year's marketing campaign. As shown in the correlation heatmap, **all correlations with Response are relatively weak**, but a few features stand out with **slightly higher influence**. The breakdown is as follows:

- **Demographics** includes **Income** (0.13) and **Enrollment_Duration** (0.17)
- **Spending Behavior** includes **MntWines** (0.25), **MntMeatProducts** (0.24), **MntGldProds** (0.14), **MntFruits** (0.13), **MntSweetProducts** (0.12), and **MntFishProducts** (0.11)
- **Purchase Preferences** includes **NumCatalogPurchases** (0.22) and **NumWebPurchases** (0.15)

Altogether, these relationships helped paint a clearer picture of which customers are more engaged, and where to focus future marketing efforts.





Data Preprocessing

Handling Outliers

Outliers can distort patterns in the data and negatively affect model performance. To address this, we applied the **Interquartile Range (IQR) method**, which identifies outliers as values that fall below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$, where IQR is the difference between the 75th and 25th percentiles.

```
1 def remove_outliers_iqr(df, column):
2     Q1 = df[column].quantile(0.25)
3     Q3 = df[column].quantile(0.75)
4     IQR = Q3 - Q1
5     lower_bound = Q1 - 1.5 * IQR
6     upper_bound = Q3 + 1.5 * IQR
7     return df[(df[column] >= lower_bound) & (df[column] <= upper_bound)]
```

Handling Outliers

Removing Outliers from All Relevant Columns (Version 1)

Using the IQR method, we removed outliers from all columns that showed significant deviation based on our distribution analysis.

This filtering reduced the dataset from **2,240 to 982 rows** , retaining only records within the normal range across multiple features. While this approach ensures cleaner data, it also limits the sample size, which may impact model generalizability.

```
<class 'pandas.core.frame.DataFrame'>
Index: 982 entries, 2 to 2237
Data columns (total 28 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Year_Birth                           982 non-null    int64
1   Income                               982 non-null    float64
2   Kidhome                              982 non-null    int64
3   Teenhome                             982 non-null    int64
4   Enrollment_Duration                 982 non-null    int64
5   Recency                             982 non-null    int64
6   MntWines                            982 non-null    int64
7   MntFruits                           982 non-null    int64
8   MntMeatProducts                     982 non-null    int64
9   MntFishProducts                     982 non-null    int64
10  MntSweetProducts                     982 non-null    int64
11  MntGoldProds                         982 non-null    int64
12  NumDealsPurchases                   982 non-null    int64
13  NumWebPurchases                     982 non-null    int64
14  NumCatalogPurchases                 982 non-null    int64
15  NumStorePurchases                   982 non-null    int64
16  NumWebVisitsMonth                   982 non-null    int64
17  Response                             982 non-null    int64
18  Complain                             982 non-null    int64
19  Education_Bachelors                 982 non-null    bool
20  Education_Basic                     982 non-null    bool
21  Education_Masters                   982 non-null    bool
22  Education_PhD                       982 non-null    bool
23  Marital_Status_Divorced             982 non-null    bool
24  Marital_Status_Married              982 non-null    bool
25  Marital_Status_Single               982 non-null    bool
26  Marital_Status_Together             982 non-null    bool
27  Marital_Status_Widow                982 non-null    bool
dtypes: bool(9), float64(1), int64(18)
memory usage: 162.1 KB
```

Handling Outliers

Removing Outliers from Year_Birth and Income Only (Version 2)

Because the first approach significantly reduced the dataset size, we applied a more conservative strategy in Version 2. Here, outlier removal was limited to only the Year_Birth and Income columns, which showed rare or unrealistic values in the distribution analysis.

This targeted filtering reduced the dataset from **2,240 to 2,229 rows**, allowing us to address key anomalies while retaining most of the data for modeling. This version offers a better balance between data quality and data volume.

```
<class 'pandas.core.frame.DataFrame'>
Index: 2229 entries, 0 to 2239
Data columns (total 28 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Year_Birth                            2229 non-null   int64
1   Income                               2229 non-null   float64
2   Kidhome                              2229 non-null   int64
3   Teenhome                             2229 non-null   int64
4   Enrollment_Duration                  2229 non-null   int64
5   Recency                              2229 non-null   int64
6   MntWines                             2229 non-null   int64
7   MntFruits                            2229 non-null   int64
8   MntMeatProducts                      2229 non-null   int64
9   MntFishProducts                      2229 non-null   int64
10  MntSweetProducts                     2229 non-null   int64
11  MntGoldProds                         2229 non-null   int64
12  NumDealsPurchases                    2229 non-null   int64
13  NumWebPurchases                      2229 non-null   int64
14  NumCatalogPurchases                 2229 non-null   int64
15  NumStorePurchases                    2229 non-null   int64
16  NumWebVisitsMonth                   2229 non-null   int64
17  Response                             2229 non-null   int64
18  Complain                             2229 non-null   int64
19  Education_Bachelors                  2229 non-null   bool
20  Education_Basic                      2229 non-null   bool
21  Education_Masters                    2229 non-null   bool
22  Education_PhD                        2229 non-null   bool
23  Marital_Status_Divorced              2229 non-null   bool
24  Marital_Status_Married               2229 non-null   bool
25  Marital_Status_Single                2229 non-null   bool
26  Marital_Status_Together              2229 non-null   bool
27  Marital_Status_Widow                 2229 non-null   bool
dtypes: bool(9), float64(1), int64(18)
memory usage: 367.9 KB
```



Handling Outliers

From this, we now have two versions of the dataset to use for modeling:

- **Version 1:** Outliers removed from all relevant columns based on distribution analysis (982 rows)
- **Version 2:** Outliers removed only from Year_Birth and Income, which showed rare or unrealistic values (2,229 rows)

These versions will be **compared during model evaluation** to determine which outlier handling strategy leads to **better performance in predicting customer responses**, helping us choose the most effective approach for our data mining goals.



Splitting the Dataset

Next, we split each version of the dataset into training and testing sets. Splitting is done first so that **scaling** and **resampling** can be applied **separately** to the training and test data.

This is important because applying preprocessing to the entire dataset before splitting can lead to data leakage, where patterns from the test set influence the training process. By splitting first and applying preprocessing independently, we ensure that the model is trained only on the training set and evaluated fairly on truly unseen data.

```
1 # Split Version 1 dataset into training and testing sets (stratify to keep class balance)
2 X_train_v1, X_test_v1, y_train_v1, y_test_v1 = train_test_split(X_v1, y_v1, test_size=0.2, stratify=y_v1, random_state=42)

1 # Split Version 2 dataset into training and testing sets (stratify to keep class balance)
2 X_train_v2, X_test_v2, y_train_v2, y_test_v2 = train_test_split(X_v2, y_v2, test_size=0.2, stratify=y_v2, random_state=42)
```

We used an **80/20 train-test split** and applied **stratification** to preserve the original label distribution in both sets, ensuring that the ratio of responders to non-responders remains consistent throughout the modeling process.

Scaling Features

After splitting the data, we applied **StandardScaler** to standardize the **numerical features**, transforming them to have a mean of 0 and a standard deviation of 1. This ensures that features are on a comparable scale, which is important for models sensitive to feature magnitude, such as Logistic Regression, K-Nearest Neighbor, and Neural Networks.

Scaling was applied **only to the input features**, not to the target variable, since the target represents class labels (0 or 1) and should remain unchanged.

To avoid data leakage, we fit the scaler on the **training set only** and then applied the same transformation to the test set separately. This ensures that the test data remains unseen during training and that the model evaluation remains valid.

```
1 # Initialize a scaler for Version 1
2 scaler_v1 = StandardScaler()
3 scaler_v1.set_output(transform='pandas')
4
5 # Fit on training set only, then transform both sets
6 X_train_v1_scaled = scaler_v1.fit_transform(X_train_v1)
7 X_test_v1_scaled = scaler_v1.transform(X_test_v1)
```

```
1 # Initialize a new scaler for Version 2
2 scaler_v2 = StandardScaler()
3 scaler_v2.set_output(transform='pandas')
4
5 # Fit on training set only, then transform both sets
6 X_train_v2_scaled = scaler_v2.fit_transform(X_train_v2)
7 X_test_v2_scaled = scaler_v2.transform(X_test_v2)
```

Balancing the Dataset

```
# Apply SMOTE to the scaled training data for Version 1
smote_v1 = SMOTE(random_state=42)
X_train_v1_balanced, y_train_v1_balanced = smote_v1.fit_resample(X_train_v1_scaled, y_train_v1)

# Apply SMOTE to the scaled training data for Version 2
smote_v2 = SMOTE(random_state=42)
X_train_v2_balanced, y_train_v2_balanced = smote_v2.fit_resample(X_train_v2_scaled, y_train_v2)
```

To address class imbalance in the training data, we applied **SMOTE (Synthetic Minority Over-sampling Technique) only on the training set**. SMOTE generates synthetic samples for the minority class by interpolating between existing samples, helping the model learn from a more balanced dataset.

It is applied **after splitting and scaling**, so that the synthetic data is generated within the same scaled feature space. Importantly, **we do not apply SMOTE to the test set**—we keep its original class distribution to reflect real-world conditions. This allows us to evaluate how well the model performs on the **naturally imbalanced data**, which is what it would encounter in actual deployment.

Final Form of Dataset Before Modeling

After completing the full data cleaning and preprocessing pipeline, the dataset is transformed into a format that is ready for modeling. At this stage, it has:

- **Irrelevant identifiers removed**, such as dropping the ID column.
- **No missing values**, with imputation applied where necessary (e.g., Income).
- **Proper data types**, including a numerical transformation of Dt_Customer.
- **Cleaned and one-hot encoded categorical features** (e.g., Education, Marital_Status).
- **Outliers handled** through two versions: one with all outliers removed (v1), and one with only Year_Birth and Income cleaned (v2).
- **Split into training and test sets** using stratified 80/20 sampling.
- **Numerical features scaled** separately for training and test sets.
- **SMOTE applied to the training set** only, balancing class distribution without affecting the test set.

The result is a fully prepared and structured dataset that is clean, balanced (in training), numerically scaled, and properly encoded—ready for robust model training and fair evaluation.

Final Form of V1 Dataset

[] 1 X_train_v1_balanced



	Year_Birth	Income	Kidhome	Teenhome	Enrollment_Duration	Recency	MntWines	MntFruits	MntMeatProducts
0	1.627699	-0.387142	0.430450	-0.909120	0.959140	-1.247439	-0.577814	-0.083819	-0.360065
1	0.360276	0.930390	0.430450	0.988910	1.537613	1.264402	0.211699	-0.688860	0.178792
2	1.537169	0.401428	0.430450	-0.909120	-1.065513	-0.215175	0.598057	1.580043	2.914529
3	-1.269268	0.225357	0.430450	0.988910	1.395153	-0.008723	-0.527419	-0.083819	-0.111362
4	-1.269268	1.442549	0.430450	0.988910	-1.255459	-0.146358	-0.628208	-0.688860	-0.608769
...	...	...	...	...	...	...	...	...	...
1425	0.337154	-0.474374	0.430450	-0.909120	0.693948	-1.151164	-0.272950	-0.373462	-0.184993
1426	0.352282	0.219113	0.430450	0.246458	0.406757	-1.109910	1.591685	2.764348	1.530446
1427	0.993987	0.270236	1.892766	-0.909120	-0.940685	-1.519549	-0.244979	-0.358943	1.338984
1428	0.658128	-0.253386	0.430450	-0.909120	0.005432	-1.188603	-0.494606	-0.544656	-0.064111
1429	0.203320	-2.055225	0.430450	-0.909120	2.352471	-1.350059	-0.639043	1.768453	-0.264288

1430 rows x 27 columns

[] 1 y_train_v1_balanced



	Response
0	0
1	0
2	0
3	0
4	0
...	...
1425	1
1426	1
1427	1
1428	1
1429	1

1430 rows x 1 columns

Final Form of V1 Dataset

[85] 1 X_test_v1_scaled



	Year_Birth	Income	Kidhome	Teenhome	Enrollment_Duration	Recency	MntWines	MntFruits	MntMeatProducts
1253	-0.545027	1.398385	-1.580877	0.98891	-0.046711	0.197730	0.900424	-0.537600	-0.235714
34	1.718229	-1.380601	-1.580877	-0.90912	-0.348898	-1.660345	-0.728997	1.126263	-0.567318
229	-1.721920	0.736995	0.430450	0.98891	0.691489	-1.385074	2.227477	-0.386340	2.002617
897	0.631866	-0.724875	0.430450	-0.90912	0.540395	-0.352811	-0.678602	0.369961	-0.318615
915	0.631866	-1.012135	0.430450	-0.90912	0.220940	-0.318402	-0.661804	-0.537600	-0.608769
...	...	...	...	...	...	...	...	...	...
79	-2.174571	-0.621182	-1.580877	-0.90912	2.366469	-1.625936	-0.594612	-0.537600	-0.857472
54	0.269745	-1.377103	0.430450	-0.90912	-1.000759	-1.625936	-0.628208	0.823742	-0.401516
1583	1.718229	-1.391345	-1.580877	-0.90912	0.954823	0.713862	-0.544217	-0.083819	-0.152813
1960	-0.726087	0.384854	0.430450	0.98891	-1.009393	1.298811	-0.493823	-0.083819	-0.525868
2146	-0.635557	-1.351451	0.430450	0.98891	0.540395	1.574081	-0.544217	-0.083819	-0.857472

197 rows × 27 columns

1 y_test_v1



	Response
1253	0
34	0
229	1
897	0
915	0
...	...
79	0
54	0
1583	0
1960	0
2146	0

197 rows × 1 columns

Final Form of V2 Dataset

[98] 1 X_train_v2_balanced



	Year_Birth	Income	Kidhome	Teenhome	Enrollment_Duration	Recency	MntWines	MntFruits
0	2.252942	0.952908	-0.823642	-0.950154	-1.796773	-0.647237	-0.066704	-0.223803
1	-0.076601	0.923761	-0.823642	0.865628	0.127293	-1.547304	0.224443	0.054468
2	1.045030	-0.026052	1.035286	0.865628	-1.471819	1.187514	-0.842107	-0.653858
3	0.095957	1.133106	1.035286	2.681410	-0.107871	1.429839	1.635617	0.838686
4	-0.335439	0.305980	-0.823642	0.865628	-1.069904	-0.508765	1.472218	-0.653858
...	...	...	...	...	...	...	...	...
3027	0.292538	1.386672	0.862769	-0.950154	0.321839	-0.195038	2.514221	-0.232666
3028	-1.034093	0.135182	-0.211557	0.865628	0.097049	-0.632038	0.327120	-0.462277
3029	1.417643	0.643647	-0.823642	-0.950154	-0.375032	-0.792556	0.439949	0.977959
3030	-0.229819	-0.383329	-0.617320	0.865628	-0.311798	-1.312662	-0.019781	-0.400857
3031	0.134529	0.671957	-0.823642	-0.950154	1.095008	0.377040	1.256463	0.880056

3032 rows × 27 columns



1 y_train_v2_balanced



	Response
0	0
1	0
2	0
3	0
4	0
...	...
3027	1
3028	1
3029	1
3030	1
3031	1

3032 rows × 1 columns

Final Form of V2 Dataset

[88] 1 X_test_v2_scaled



	Year_Birth	Income	Kidhome	Teenhome	Enrollment_Duration	Recency	MntWines	MntFruits
2060	-1.457071	1.146434	-0.823642	-0.950154	-1.352100	1.499075	-0.304376	0.231550
554	-0.853116	0.962867	-0.823642	-0.950154	-0.189109	-0.854945	-0.233074	-0.072019
598	-0.335439	0.974877	-0.823642	-0.950154	-0.180558	-0.785709	0.358133	0.585712
1423	0.613634	-0.119983	-0.823642	0.865628	-2.049039	0.495155	-0.666825	-0.628560
2049	-0.853116	0.956276	-0.823642	0.865628	-0.603852	1.464457	1.032526	-0.223803
...	...	...	...	...	...	...	...	...
1788	-1.284513	1.226793	-0.823642	-0.950154	0.576241	1.049042	0.625514	0.762794
746	-0.939395	1.372671	-0.823642	0.865628	0.388110	-0.578001	0.325454	-0.527371
694	1.217589	-1.052026	1.035286	-0.950154	-1.352100	-0.647237	-0.892612	-0.577966
415	1.562707	1.450784	-0.823642	-0.950154	-0.984390	-1.062652	-0.423211	-0.072019
452	-0.766836	1.740977	-0.823642	-0.950154	1.238975	-1.028034	0.117491	-0.021424

446 rows × 27 columns

1 y_test_v2



	Response
2060	0
554	0
598	0
1423	0
2049	0
...	...
1788	0
746	0
694	0
415	0
452	1

446 rows × 1 columns



Model Building

Classification Technique

Goal: Predict whether a customer will accept a VIP membership offer → Binary classification problem (Target variable is Response: 1 = accept, 0 = not accept)

Why Classification?

- Predict likelihood of acceptance → rank & target customers
- Understand key drivers of decisions → supports strategic insights

Models Used

- **Logistic Regression** – Simple, interpretable baseline
- **Decision Tree** – Rule-based, transparent
- **Random Forest** – Reduces overfitting, handles noise
- **Extra Trees** – More randomness, better generalization
- **AdaBoost** – Boosts misclassified cases, improves tough predictions
- **Gradient Boosting** – Strong performance, optimized sequentially
- **Naive Bayes** – Fast, handles high-dimensional data
- **K-Nearest Neighbors (KNN)** – Simple, proximity-based
- **MLPClassifier** – Neural network for complex patterns

Why multiple models?

- To compare strengths, interpretability, and predictive power → find best fit for business needs



Model Evaluation

Success Metrics

In our analysis, we use Precision, Recall, F1 Score, ROC AUC, and the Confusion Matrix to evaluate model performance. These metrics are particularly relevant to our goal of predicting the likelihood of customers accepting the VIP membership offer and identifying the factors that influence their decision.

1. **Precision** measures the accuracy of positive predictions, indicating how many of the customers predicted to accept the offer actually accepted it.
2. **Recall** evaluates how many of the actual positive instances (i.e., customers who accepted the offer) were correctly identified by the model.
3. The **F1 Score** is the harmonic mean of precision and recall, offering a balanced metric when both false positives and false negatives are important.
4. **ROC AUC** evaluates the model's ability to distinguish between positive and negative classes. A higher ROC AUC indicates a model with a better ability to correctly classify customers as likely or unlikely to accept the offer.
5. The **Confusion Matrix** visualizes the true positives, false positives, true negatives, and false negatives in a tabular form.



Model Performance: V1 vs. V2

To compare modeling outcomes, we tested two approaches to handling outliers. Version 1 removed outliers from all columns, resulting in a smaller but cleaner dataset. In contrast, Version 2 removed outliers only from Year_Birth and Income, preserving most of the data.

From the results, clear patterns emerged:

- Version 1 tended to show **slightly higher precision** in some models, likely because the cleaner data made the models more cautious in predicting positive responses. However, this came at a cost—these models often missed more actual positives.
- On the other hand, Version 2 consistently delivered **higher recall and F1 scores**, meaning it was better at identifying true positives and achieving a more balanced prediction overall.
- **ROC AUC scores remained high across both versions**, but Version 2 models showed more consistent separation between responders and non-responders.

Therefore, while Version 1 prioritized cleanliness, the reduced dataset limited learning. Meanwhile, Version 2, by keeping more data and focusing only on key outliers, produced stronger, more generalizable models.



Overview of Model Performance (V1 vs v2)

Version 1 Dataset				
Model	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	0.3871	0.6667	0.4898	0.8534
Naive Bayes	0.2167	0.7222	0.3333	0.7795
KNN	0.1957	0.5000	0.2812	0.7354
Decision Tree	0.3500	0.3889	0.3684	0.6581
Random Forest	0.4667	0.3889	0.4242	0.8813
Extra Trees	0.3571	0.2778	0.3125	0.8032
AdaBoost	0.3929	0.6111	0.4783	0.8894
Gradient Boosting	0.5263	0.5556	0.5405	0.9207
Neural Network	0.4118	0.3889	0.4000	0.8782

Version 2 Dataset				
Model	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	0.4094	0.7761	0.5361	0.8767
Naive Bayes	0.2553	0.7164	0.3765	0.7438
KNN	0.3041	0.6716	0.4186	0.7613
Decision Tree	0.4304	0.5075	0.4658	0.6924
Random Forest	0.6471	0.4925	0.5593	0.8950
Extra Trees	0.6667	0.4776	0.5565	0.8895
AdaBoost	0.4737	0.6716	0.5556	0.8677
Gradient Boosting	0.6056	0.6418	0.6232	0.9047
Neural Network	0.5152	0.5075	0.5113	0.8661

Confusion Matrix: V1 vs. V2

Meanwhile, to better understand how each model handled predictions, we analyzed their confusion matrices.

In Version 1, where outliers were removed from all columns, models tended to be more conservative. Most had **low false positives (FP) and high true negatives (TN)**, which led to slightly higher precision in some cases. However, they also produced **fewer true positives (TP)**, meaning they missed many customers who would have accepted the offer.

In contrast, Version 2 models—built on a larger dataset with selective outlier removal—achieved **significantly higher true positives** across the board. This improvement came with a moderate increase in false positives, but it allowed the models to **capture more actual responders**, improving recall and F1 scores.

For example:

- **Gradient Boosting (v1):** TP = 10, FP = 9
- **Gradient Boosting (v2):** TP = 43, FP = 28

In conclusion, Version 1 models were more cautious but often missed potential responders. On the other hand, Version 2 struck a better balance—identifying more true positives while keeping false positives reasonable, supporting its superior F1 and recall metrics.

Overview of Confusion Matrices (V1 vs V2)

Version 1 Dataset				
Model	True Negative	False Positive	False Negative	True Positive
Logistic Regression	160	19	6	12
Naive Bayes	132	47	5	13
KNN	142	37	9	9
Decision Tree	166	13	11	7
Random Forest	171	8	11	7
Extra Trees	170	9	13	5
AdaBoost	162	17	7	11
Gradient Boosting	170	9	8	10
Neural Network	169	10	11	7

Version 2 Dataset				
Model	True Negative	False Positive	False Negative	True Positive
Logistic Regression	304	75	15	52
Naive Bayes	239	140	19	48
KNN	276	103	22	45
Decision Tree	334	45	33	34
Random Forest	361	18	34	33
Extra Trees	363	16	35	32
AdaBoost	329	50	22	45
Gradient Boosting	351	28	24	43
Neural Network	347	32	33	34

Ranking of Model Performance

Ranking				
Rank	Model	F1 Score	ROC AUC	Avg Percentile
1	Gradient Boosting (v2)	0.6232	0.9047	0.9705
2	Random Forest (v2)	0.5593	0.895	0.9115
3	Gradient Boosting (v1)	0.5405	0.9207	0.8825
4	Extra Trees (v2)	0.5565	0.8895	0.8530
5	AdaBoost (v2)	0.5556	0.8677	0.6765
6	Logistic Regression (v2)	0.5361	0.8767	0.6470
7	AdaBoost (v1)	0.4783	0.8894	0.6470
8	Neural Network (v2)	0.5113	0.8661	0.5590
9	Random Forest (v1)	0.4242	0.8813	0.5590

Ranking of Model Performance

Ranking				
Rank	Model	F1 Score	ROC AUC	Avg Percentile
10	Logistic Regression (v1)	0.4898	0.8534	0.5000
11	Neural Network (v1)	0.4	0.8782	0.4705
12	KNN (v2)	0.4186	0.7613	0.2940
13	Decision Tree (v2)	0.4658	0.6924	0.2650
14	Naive Bayes (v2)	0.3765	0.7438	0.2055
15	Naive Bayes (v1)	0.3333	0.7795	0.2060
16	Extra Trees (v1)	0.3125	0.8032	0.2060
17	Decision Tree (v1)	0.3684	0.6581	0.0880
18	KNN (v1)	0.2812	0.7354	0.0590



The Best-Performing Model

To determine the best-performing model, we focused on F1 Score and ROC AUC as our primary evaluation metrics. These were chosen based on the nature of our problem:

- **F1 Score** provides a balance between precision and recall, which is essential in our case where both false positives (wasting marketing resources) and false negatives (missing potential conversions) are costly.
- **ROC AUC** measures the model's ability to distinguish between the two classes (responders and non-responders), making it especially relevant for assessing overall classification performance regardless of threshold.

To rank all models fairly and consistently, we used the **average percentile ranking method** , which converts each model's F1 Score and ROC AUC into percentile ranks and then averages them. This method helps normalize differences across metrics and ensures that no single metric disproportionately affects the ranking.



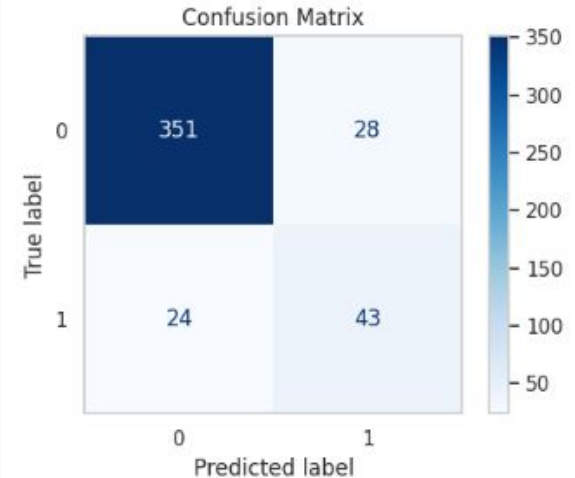
The Best-Performing Model

Based on this average percentile ranking, **Gradient Boosting using Dataset Version 2** emerged as the best-performing model. It achieved:

- The **highest F1 Score** of **0.6232**, indicating the best balance between precision and recall.
- A **strong ROC AUC** of 0.9047, demonstrating excellent class separation.
- An **average percentile score** of **0.9705**, the highest among all models evaluated.

This result is further supported by the Confusion Matrix, where the model achieved **43 true positives (TP)** and only **28 false positives (FP)**. This shows the best balance between capturing the true positive responses and minimizing false positives, reinforcing its top F1 and ROC AUC scores.

Performance Summary: Gradient Boosting (v2)
Precision : 0.6056
Recall : 0.6418
F1 Score : 0.6232
ROC AUC : 0.9047



Feature Selection

To simplify the best-performing Gradient Boosting model, we used **Sequential Feature Selection (SFS) with forward selection and cross-validation**, optimizing for ROC AUC. The process reduced model complexity while preserving performance, **selecting 13 key features** across demographics, engagement, and purchase behavior.

These selected features improve interpretability and highlight that both who the customer is and how they interact with the company matter in predicting offer acceptance.

Number of selected features: 13

Selected features:

1. Income
2. Teenhome
3. Enrollment_Duration
4. Recency
5. Mntwines
6. NumDealsPurchases
7. NumWebPurchases
8. NumCatalogPurchases
9. NumStorePurchases
10. NumWebVisitsMonth
11. Education_PhD
12. Marital_Status_Married
13. Marital_Status_Together

Feature Selection

After applying Sequential Feature Selection (SFS), the Gradient Boosting model achieved:

- **Precision:** 0.5844
- **Recall:** 0.6716
- **F1 Score:** 0.6250
- **ROC AUC:** 0.8955

Performance remained close to the original model (F1: 0.6232, AUC: 0.9047), showing SFS is effective for maintaining accuracy while reducing inputs.

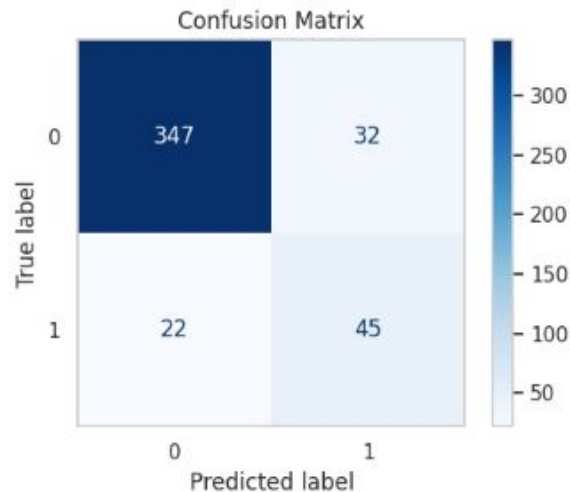
Performance Summary: Gradient Boosting (with SFS)

Precision : 0.5844

Recall : 0.6716

F1 Score : 0.6250

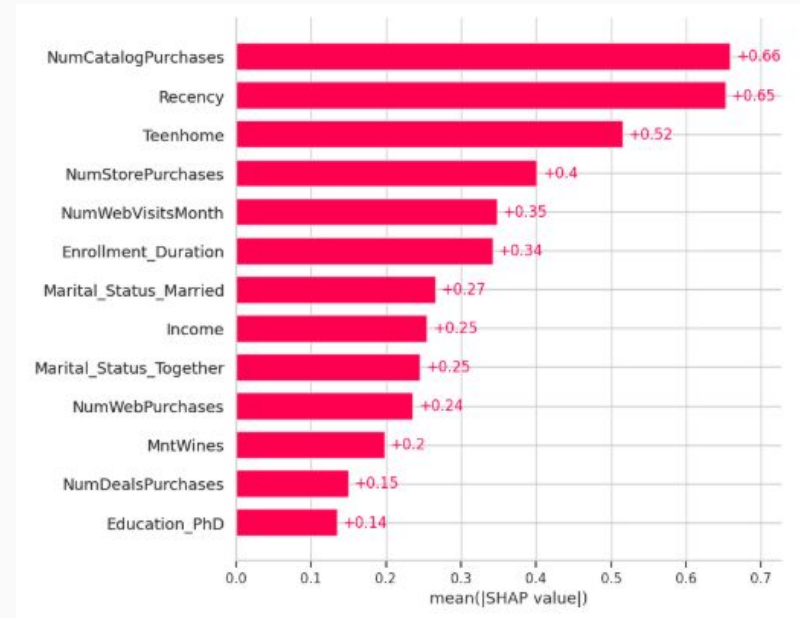
ROC AUC : 0.8955



SHAP Analysis

SHAP (SHapley Additive exPlanations) was used to interpret the Gradient Boosting model, focusing on the 13 features selected via SFS. SHAP highlights how much each feature contributes to predicting VIP membership acceptance.

- Top Features
 - **NumCatalogPurchases** and **Recency** are the strongest drivers—frequent catalog buyers and recent purchasers are more likely to accept the offer.
 - **Teenhome** suggests household composition also influences decisions.
- Mid-level Contributors:
 - **NumStorePurchases**, **NumWebVisitsMonth**, and **Enrollment_Duration** point to customer engagement and loyalty over time.
- Supporting Features:
 - **Marital_Status**, **Income**, and **Education_PhD** show that demographics contribute, but less than behavioral data.



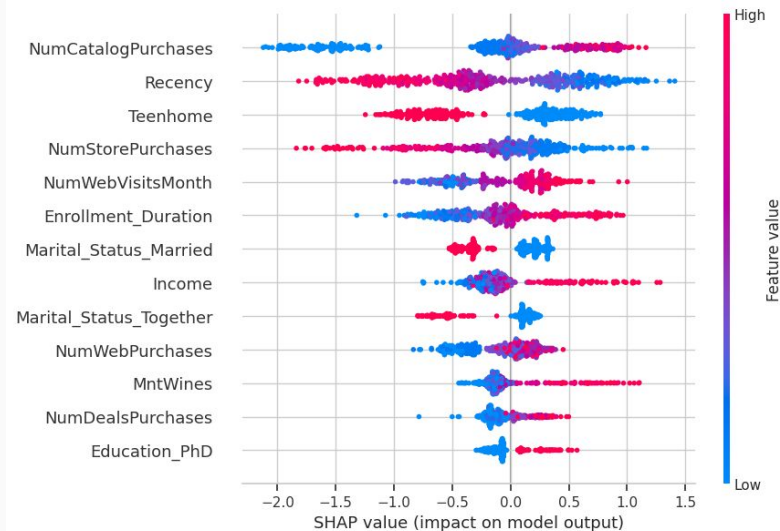
SHAP Analysis

This SHAP beeswarm plot shows how each selected feature influences individual predictions. Each dot = a customer.

- X-axis (SHAP value): Right = pushes toward acceptance, Left = pushes away
- Color: Red = high feature value, Blue = low value

Key Insights:

- **NumCatalogPurchases** : More catalog purchases → more likely to accept
- **Recency** : More recent activity → higher chance of acceptance
- **Teenhome** : More teenagers at home → less likely to accept
- **NumStorePurchases** : Frequent in-store shoppers → less responsive
- **NumWebVisitsMonth** : More visits → more likely to accept
- **Enrollment_Duration** : Longer tenure → slightly more likely
- **Marital_Status (Married/Together)** : Lean left → less likely to accept
- **Income & MntWines** : Higher values → more likely to accept
- **NumDealsPurchases** & **Education_PhD** : Slight positive influence
- Behavioral features (catalog, web, deals) are stronger drivers for acceptance than demographics. The plot reveals clear patterns to guide targeted strategies.





02

Changes and Improvement s



Pipeline Summary

01

Data Description

02

Data Cleaning

03

Exploratory Data Analysis

04

Data Preprocessing

05

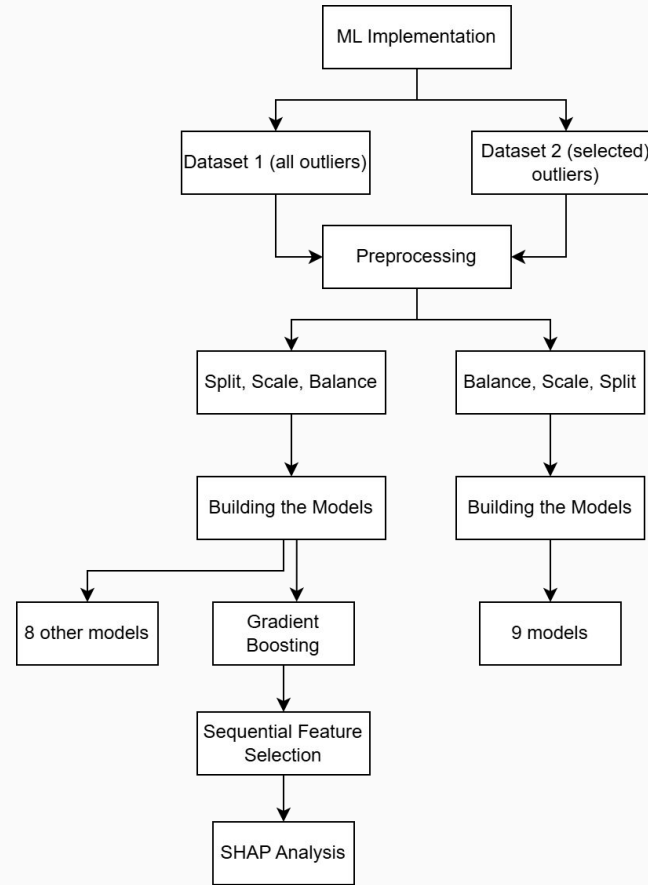
Model Building

06

Model Evaluation



Pipeline Diagram



Correcting Error in One-Hot Encoding

Initially, we encoded all categories without dropping the first column, which may have introduced multicollinearity.

After correction, we used `drop_first=True` to avoid the dummy variable trap, ensuring each categorical variable contributes unique, non-redundant information to the model.

Before Correction

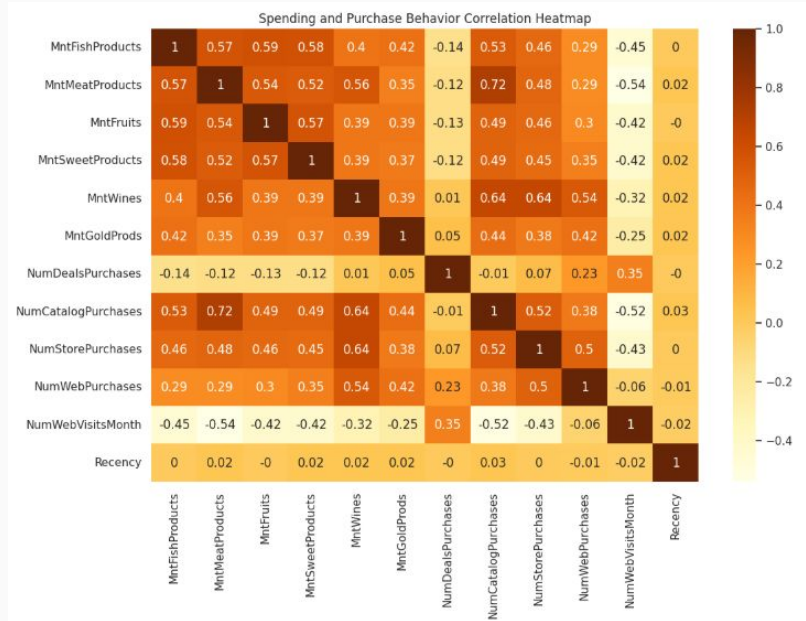
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 28 columns):
#   Column                               Non-Null Count  Dtype
---  -
0   Year_Birth                           2240 non-null   int64
1   Income                               2240 non-null   float64
2   Kidhome                              2240 non-null   int64
3   Teenhome                             2240 non-null   int64
4   Enrollment_Duration                 2240 non-null   int64
5   Recency                             2240 non-null   int64
6   MntWines                            2240 non-null   int64
7   MntFruits                           2240 non-null   int64
8   MntMeatProducts                     2240 non-null   int64
9   MntFishProducts                     2240 non-null   int64
10  MntSweetProducts                    2240 non-null   int64
11  MntGoldProds                        2240 non-null   int64
12  NumDealsPurchases                   2240 non-null   int64
13  NumWebPurchases                     2240 non-null   int64
14  NumCatalogPurchases                 2240 non-null   int64
15  NumStorePurchases                   2240 non-null   int64
16  NumWebVisitsMonth                   2240 non-null   int64
17  Response                             2240 non-null   int64
18  Complain                             2240 non-null   int64
19  Education_Bachelors                 2240 non-null   bool
20  Education_Basic                     2240 non-null   bool
21  Education_Masters                   2240 non-null   bool
22  Education_PhD                       2240 non-null   bool
23  Marital_Status_Divorced              2240 non-null   bool
24  Marital_Status_Married               2240 non-null   bool
25  Marital_Status_Single                2240 non-null   bool
26  Marital_Status_Together              2240 non-null   bool
27  Marital_Status_Widow                 2240 non-null   bool
dtypes: bool(9), float64(1), int64(18)
memory usage: 352.3 KB
```

After Correction

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 26 columns):
#   Column                               Non-Null Count  Dtype
---  -
0   Year_Birth                           2240 non-null   int64
1   Income                               2240 non-null   float64
2   Kidhome                              2240 non-null   int64
3   Teenhome                             2240 non-null   int64
4   Enrollment_Duration                 2240 non-null   int64
5   Recency                             2240 non-null   int64
6   MntWines                            2240 non-null   int64
7   MntFruits                           2240 non-null   int64
8   MntMeatProducts                     2240 non-null   int64
9   MntFishProducts                     2240 non-null   int64
10  MntSweetProducts                    2240 non-null   int64
11  MntGoldProds                        2240 non-null   int64
12  NumDealsPurchases                   2240 non-null   int64
13  NumWebPurchases                     2240 non-null   int64
14  NumCatalogPurchases                 2240 non-null   int64
15  NumStorePurchases                   2240 non-null   int64
16  NumWebVisitsMonth                   2240 non-null   int64
17  Response                             2240 non-null   int64
18  Complain                             2240 non-null   int64
19  Education_Basic                     2240 non-null   bool
20  Education_Masters                   2240 non-null   bool
21  Education_PhD                       2240 non-null   bool
22  Marital_Status_Married               2240 non-null   bool
23  Marital_Status_Single                2240 non-null   bool
24  Marital_Status_Together              2240 non-null   bool
25  Marital_Status_Widow                 2240 non-null   bool
dtypes: bool(9), float64(1), int64(18)
memory usage: 347.9 KB
```

Highly Correlated Features

Several spending and purchase-related features exhibit strong positive correlations. In particular, **MntMeatProducts** is highly correlated with **NumCatalogPurchases** (0.72), suggesting that customers who spend more on meat products also tend to purchase more through catalogs.



Dropping NumCatalogPurchases

This may indicate overlapping behavioral patterns, which could lead to multicollinearity if both variables are used in the model without adjustment.

Among the two, we dropped **NumCatalogPurchases** as it showed **lower variance** and a **weaker correlation** with the response variable compared to **MntMeatProducts**.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 25 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Year_Birth                            2240 non-null   int64
1   Income                                2240 non-null   float64
2   Kidhome                               2240 non-null   int64
3   Teenhome                              2240 non-null   int64
4   Enrollment_Duration                  2240 non-null   int64
5   Recency                              2240 non-null   int64
6   MntWines                             2240 non-null   int64
7   MntFruits                            2240 non-null   int64
8   MntMeatProducts                      2240 non-null   int64
9   MntFishProducts                      2240 non-null   int64
10  MntSweetProducts                     2240 non-null   int64
11  MntGoldProds                         2240 non-null   int64
12  NumDealsPurchases                    2240 non-null   int64
13  NumWebPurchases                      2240 non-null   int64
14  NumStorePurchases                    2240 non-null   int64
15  NumWebVisitsMonth                    2240 non-null   int64
16  Response                             2240 non-null   int64
17  Complain                             2240 non-null   int64
18  Education_Basic                      2240 non-null   bool
19  Education_Masters                    2240 non-null   bool
20  Education_PhD                        2240 non-null   bool
21  Marital_Status_Married               2240 non-null   bool
22  Marital_Status_Single                2240 non-null   bool
23  Marital_Status_Together              2240 non-null   bool
24  Marital_Status_Widow                 2240 non-null   bool
dtypes: bool(7), float64(1), int64(17)
memory usage: 330.4 KB
```

Reapplying the Models

After dropping NumCatalogPurchases, the dataset increased slightly to 999 instances, compared to 982 in the earlier version. The second version, with selective outlier removal, remained unchanged in row count.

With the updated dataset in place, we then applied the same preprocessing pipeline from the previous report and proceeded to build the models, using the same set of classification algorithms.

```
<class 'pandas.core.frame.DataFrame'>
Index: 999 entries, 2 to 2237
Data columns (total 25 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Year_Birth                            999 non-null    int64
 1   Income                                999 non-null    float64
 2   Kidhome                               999 non-null    int64
 3   Teenhome                              999 non-null    int64
 4   Enrollment_Duration                  999 non-null    int64
 5   Recency                              999 non-null    int64
 6   MntWines                             999 non-null    int64
 7   MntFruits                            999 non-null    int64
 8   MntMeatProducts                      999 non-null    int64
 9   MntFishProducts                      999 non-null    int64
10   MntSweetProducts                     999 non-null    int64
11   MntGoldProds                         999 non-null    int64
12   NumDealsPurchases                    999 non-null    int64
13   NumWebPurchases                      999 non-null    int64
14   NumStorePurchases                    999 non-null    int64
15   NumWebVisitsMonth                    999 non-null    int64
16   Response                             999 non-null    int64
17   Complain                             999 non-null    int64
18   Education_Basic                      999 non-null    bool
19   Education_Masters                    999 non-null    bool
20   Education_PhD                        999 non-null    bool
21   Marital_Status_Married                999 non-null    bool
22   Marital_Status_Single                 999 non-null    bool
23   Marital_Status_Together               999 non-null    bool
24   Marital_Status_Widow                  999 non-null    bool
dtypes: bool(7), float64(1), int64(17)
memory usage: 155.1 KB
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 25 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Year_Birth                            2240 non-null   int64
 1   Income                                2240 non-null   float64
 2   Kidhome                               2240 non-null   int64
 3   Teenhome                              2240 non-null   int64
 4   Enrollment_Duration                  2240 non-null   int64
 5   Recency                              2240 non-null   int64
 6   MntWines                             2240 non-null   int64
 7   MntFruits                            2240 non-null   int64
 8   MntMeatProducts                      2240 non-null   int64
 9   MntFishProducts                      2240 non-null   int64
10   MntSweetProducts                     2240 non-null   int64
11   MntGoldProds                         2240 non-null   int64
12   NumDealsPurchases                    2240 non-null   int64
13   NumWebPurchases                      2240 non-null   int64
14   NumStorePurchases                    2240 non-null   int64
15   NumWebVisitsMonth                    2240 non-null   int64
16   Response                             2240 non-null   int64
17   Complain                             2240 non-null   int64
18   Education_Basic                      2240 non-null   bool
19   Education_Masters                    2240 non-null   bool
20   Education_PhD                        2240 non-null   bool
21   Marital_Status_Married                2240 non-null   bool
22   Marital_Status_Single                 2240 non-null   bool
23   Marital_Status_Together               2240 non-null   bool
24   Marital_Status_Widow                  2240 non-null   bool
dtypes: bool(7), float64(1), int64(17)
memory usage: 330.4 KB
```

Updated Model Performance

In both experiments, Version 2 outperforms Version 1 in F1 Score, confirming the benefits of selective outlier removal. Compared to the earlier run, the current results show lower F1 Scores but more stable ROC AUC, indicating improved ranking consistency despite reduced threshold-level performance.

Model	Version 1 Dataset				Version 2 Dataset			
	Precision	Recall	F1 Score	ROC AUC	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	26.19%	61.11%	36.67%	85.99%	40.91%	80.60%	54.27%	86.75%
Naive Bayes	20.69%	66.67%	31.58%	83.21%	24.62%	71.64%	36.64%	74.80%
KNN	27.66%	72.22%	40.00%	75.67%	30.28%	64.18%	41.15%	77.51%
Decision Tree	31.03%	50.00%	38.30%	71.83%	41.11%	55.22%	47.13%	70.32%
Random Forest	46.15%	33.33%	38.71%	89.18%	56.60%	44.78%	50.00%	88.61%
Extra Trees	33.33%	16.67%	22.22%	85.12%	61.70%	43.28%	50.88%	88.66%
AdaBoost	46.15%	66.67%	54.55%	91.27%	39.81%	61.19%	48.24%	84.45%
Gradient Boosting	45.00%	50.00%	47.37%	89.96%	61.33%	68.66%	64.79%	89.80%
Neural Network	28.57%	22.22%	25.00%	85.87%	52.86%	55.22%	54.01%	85.48%

Updated Confusion Matrices

In both experiments, Version 2 yields more true positives but also more false positives, indicating higher recall but at the cost of precision. This trade-off is more pronounced in the current results, reinforcing Version 2's tendency to favor sensitivity over specificity.

Model	Version 1 Dataset				Version 2 Dataset			
	TN	FP	TP	FN	TN	FP	TP	FN
Logistic Regression	151	31	7	11	301	78	13	54
Naive Bayes	136	46	6	12	232	147	19	48
KNN	148	34	5	13	280	99	24	43
Decision Tree	162	20	9	9	326	53	30	37
Random Forest	175	7	12	6	356	23	37	30
Extra Trees	176	6	15	3	361	18	38	29
AdaBoost	168	14	6	12	317	62	26	41
Gradient Boosting	171	11	9	9	350	29	21	46
Neural Network	172	10	14	4	346	33	30	37

New Pipeline: Model Performance

The new pipeline applies SMOTE before splitting the data, which causes data leakage by allowing synthetic samples to influence both training and test sets—leading to unrealistically high performance metrics. In contrast, the original pipeline avoids this issue by applying SMOTE only after splitting, yielding more reliable and valid evaluation results.

Model	Version 1 Dataset				Version 2 Dataset			
	Precision	Recall	F1 Score	ROC AUC	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	90.76%	91.76%	91.26%	96.30%	84.74%	84.96%	84.85%	92.96%
Naive Bayes	86.01%	91.21%	88.53%	93.74%	69.55%	69.92%	69.74%	80.38%
KNN	89.16%	99.45%	94.03%	98.11%	83.76%	93.93%	88.56%	94.71%
Decision Tree	90.66%	90.66%	90.66%	90.66%	87.40%	86.02%	86.70%	87.10%
Random Forest	95.24%	98.90%	97.04%	98.70%	92.48%	97.36%	94.86%	98.57%
Extra Trees	95.16%	97.25%	96.20%	97.25%	93.06%	95.51%	94.27%	98.10%
AdaBoost	94.09%	96.15%	95.11%	97.89%	86.90%	85.75%	86.32%	94.23%
Gradient Boosting	94.18%	97.80%	95.96%	98.13%	89.23%	91.82%	90.51%	96.57%
Neural Network	94.12%	96.70%	95.39%	98.70%	90.75%	93.14%	91.93%	97.14%

Overview of Confusion Matrices

The confusion matrices in the new pipeline show unnaturally low false positives and false negatives, indicating that the model is likely memorizing patterns due to data leakage from pre-split SMOTE. In comparison, the original pipeline's matrices reflect a more realistic distribution of errors, supporting the credibility of its performance metrics.

Model	Version 1 Dataset				Version 2 Dataset			
	TN	FP	TP	FN	TN	FP	TP	FN
Logistic Regression	165	17	15	167	321	58	57	322
Naive Bayes	155	27	16	166	263	116	114	265
KNN	160	22	1	181	310	69	23	356
Decision Tree	165	17	17	165	332	47	53	326
Random Forest	173	9	2	180	349	30	10	369
Extra Trees	173	9	5	177	352	27	17	362
AdaBoost	171	11	7	175	330	49	54	325
Gradient Boosting	171	11	4	178	337	42	31	348
Neural Network	171	11	6	176	343	36	26	353

Ranking of Model Performance

Models were ranked using a weighted score (40% F1, 60% ROC AUC) to reflect both balance and discrimination. Gradient Boosting (v2) ranked highest with a score of 79.80%, followed by AdaBoost (v1) and Logistic Regression (v2).

Rank	Model	F1 Score	ROC AUC	Avg Percentile
1	Gradient Boosting (v2)	64.79%	89.80%	79.80%
2	AdaBoost (v1)	54.55%	91.27%	76.58%
3	Logistic Regression (v2)	54.27%	86.75%	73.76%
4	Extra Trees (v2)	50.88%	88.66%	73.55%
5	Random Forest (v2)	50.00%	88.61%	73.17%
6	Gradient Boosting (v1)	47.37%	89.96%	72.61%
7	Neural Network (v2)	54.01%	85.48%	70.89%
8	AdaBoost (v2)	48.24%	84.45%	70.17%
9	Extra Trees (v1)	22.22%	85.12%	59.98%

Ranking of Model Performance

Rank	Model	F1 Score	ROC AUC	Avg Percentile
10	Random Forest (v1)	38.71%	89.18%	57.10%
11	Logistic Regression (v1)	36.67%	85.99%	56.78%
12	Neural Network (v1)	25.00%	85.87%	51.52%
13	KNN (v2)	41.15%	77.51%	61.97%
14	Decision Tree (v2)	47.13%	70.32%	60.60%
15	KNN (v1)	40.00%	75.67%	61.40%
16	Decision Tree (v1)	38.30%	71.83%	60.42%
17	Naive Bayes (v1)	31.58%	83.21%	62.56%
18	Naive Bayes (v2)	36.64%	74.80%	59.50%

Feature Selection

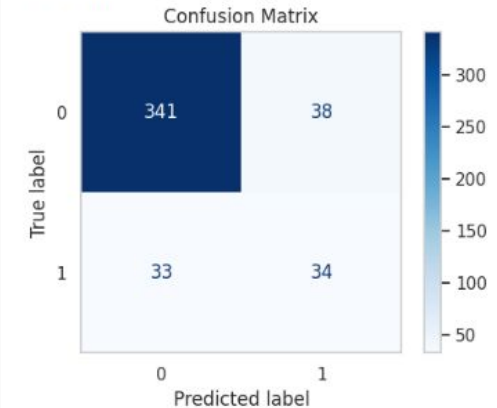
Applying Sequential Feature Selection (SFS) to Gradient Boosting resulted in a model using 12 key features, but the overall performance declined, with the F1 Score dropping to 48.92%.

Despite maintaining a relatively high ROC AUC of 85.26%, the confusion matrix shows increased misclassifications, indicating reduced effectiveness in balancing precision and recall.

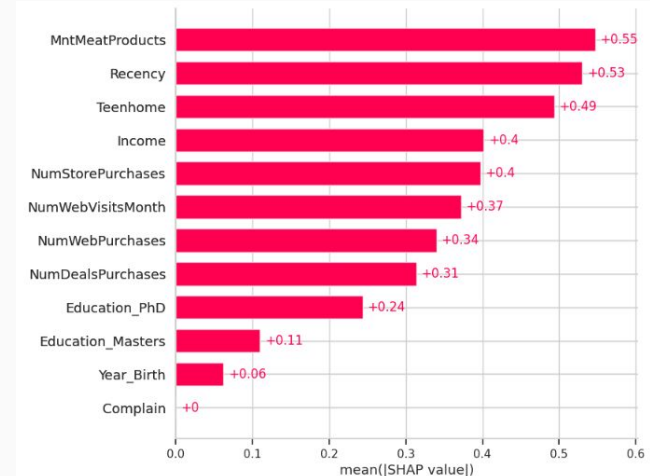
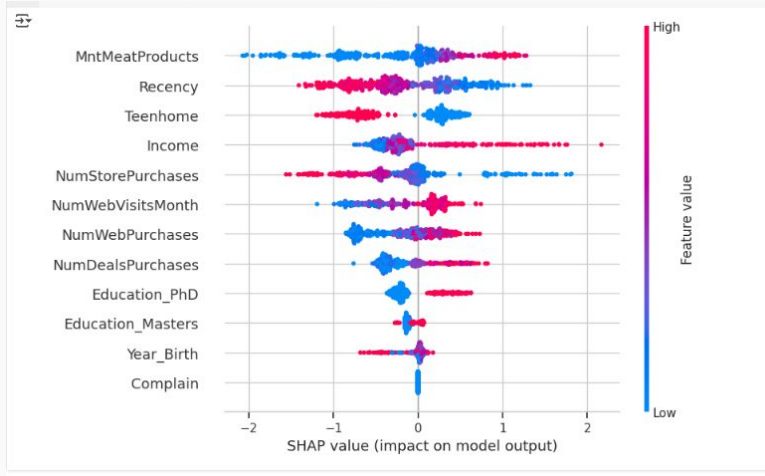
```
Number of selected features: 12
Selected features:
1. Year_Birth
2. Income
3. Teenhome
4. Recency
5. MntMeatProducts
6. NumDealsPurchases
7. NumWebPurchases
8. NumStorePurchases
9. NumWebVisitsMonth
10. Complain
11. Education_Masters
12. Education_PhD
```

```
Performance Summary: Gradient Boosting (with SFS)
Precision : 0.4722
Recall    : 0.5075
F1 Score  : 0.4892
ROC AUC   : 0.8526
```

```
[[341  38]
 [ 33  34]]
```



SHAP Analysis



These SHAP plots explain how each feature influences the Gradient Boosting model's predictions: the left plot shows the impact of individual feature values (colored by magnitude), while the right plot summarizes the average importance of each feature. Features like MntMeatProducts, Recency, and Teenhome have the highest positive impact on the model output, while features like Complain and Year_Birth contribute minimally.



03

Final Evaluation



Final Evaluation

To address the company's key business challenges:

- High marketing costs
- Low conversion rates
- Lack of customer targeting strategy

The **Gradient Boosting trained on Dataset Version 2** delivered the most effective results. With the highest **F1 Score of 0.6232**, it reflects a strong balance between precision and recall. An **ROC AUC of 0.9047** also shows excellent ability to distinguish between responders and non-responders.

With **43 true positives** and only **28 false positives** in the confusion matrix, this model reduces wasted calls to uninterested customers while capturing a significant portion of likely buyers. By using this model, the business can adopt a **data-driven targeting strategy, focus sales efforts on the most promising leads, and significantly improve campaign efficiency**, all while lowering the cost and effort involved in manual outreach.



04

Insights and Reflections



Insights and Reflections

1. Customer engagement is multidimensional

- **No single factor fully predicts whether a customer will respond to a campaign** .

While spending behavior and digital activity (e.g., web and catalog purchases) show moderate influence, demographic factors like income and enrollment duration also play a role.

2. More Data ≠ Better Data

- **Over-cleaning or removing too many outliers (Version 1) led to weaker performance** .

Preserving the data's variability (as in Version 2) produced better generalization and insights, highlighting the importance of context in data preparation.

3. Marketing should be more targeted

- With only ~15% of customers accepting the offer and <1% filing complaints, the challenge lies in **identifying the right customers, not reaching more people** . This analysis can inform more efficient, personalized campaigns going forward.